← Retour index Python Niveau II

Python Niveau II — Syllabus

POO avancée • Décorateurs • Coroutines • async/await

Durée recommandée : **2 jours (14h)** — niveau intermédiaire.

Objectif général

Transformer une pratique Python « script » en une pratique « application » : code structuré, réutilisable, testable et capable de gérer des I/O concurrentes proprement.

Progression pédagogique (du simple au complexe)

Étape 1 — Structurer le code

Passer d'un script procédural à un modèle orienté objet clair.

Étape 2 — Factoriser les comportements transverses

Appliquer des décorateurs pour logs, validation, retry.

Étape 3 — Concurrence asynchrone

Comprendre l'event loop et piloter des coroutines.

Étape 4 — Fiabiliser en conditions réelles

Gérer timeouts, annulations, limites de concurrence et erreurs.

Étape 5 — Intégrer

Assembler objet + décorateurs + async dans un mini-service cohérent.

Définitions essentielles

- **Classe** : plan de construction d'objets portant état et comportements.
- **Invariant** : règle qui doit rester vraie pendant la vie d'un objet.
- **Décorateur** : fonction qui enrichit une autre fonction sans modifier son code interne.
- **Coroutine** : fonction asynchrone suspendable/reprenable avec `await`.
- **Event loop** : ordonnanceur des tâches asynchrones.

Notions théoriques supplémentaires

- **Data model Python** : protocole d'objets piloté par des méthodes spéciales (dunder).
- **Méthodes magiques** : `__repr__`, `__str__`, `__len__`, `__iter__`, `__eq__`, `__lt__`, `__hash__`, `__call__`.
- **Protocole de contexte** : `__enter__` / `__exit__` pour gérer proprement les ressources.
- **Descriptors et properties** : encapsulation fine via `@property` et validation d'accès.
- **MRO** : ordre de résolution des méthodes en héritage multiple.
- **Protocoles de typage** : interfaces structurelles avec `typing.Protocol`.

Extraits de code de référence

POO + invariants

```
from dataclasses import dataclass

@dataclass
class Compte:
    id: str
    solde: float

    def __post_init__(self) -> None:
        if self.solde < 0:
            raise ValueError("solde invalide")
```

Méthodes magiques (dunder)

```
class Panier:
    def __init__(self, items: list[str]) -> None:
        self.items = items
```

```
    def __len__(self) -> int:
        return len(self.items)

    def __iter__(self):
        return iter(self.items)

    def __repr__(self) -> str:
        return f"Panier(items={self.items!r})"
```

Décorateur

```
import functools

def log_call(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        print(f"call {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper
```

Async/await

```
import asyncio

async def fetch_one(i: int) -> str:
    await asyncio.sleep(0.2)
    return f"ok-{i}"

async def main() -> None:
    results = await asyncio.gather(*(fetch_one(i) for i in range(5)))
    print(results)
```

Compétences visées

- Concevoir des classes métier robustes (invariants, encapsulation, composition).
- Écrire des décorateurs lisibles et sûrs sans effets de bord.
- Orchestrer des appels I/O concurrents avec `asyncio`.
- Diagnostiquer les anti-patterns (blocage event loop, couplage fort, exceptions silencieuses).

- Produire un code maintenable avec tests ciblés.

Public cible et prérequis

- Public : développeurs Python juniors/intermédiaires, profils data techniques.
- Prérequis : fonctions, modules, exceptions, manipulations de fichiers et notions de tests.

Livrables stagiaire

- Mini-service asynchrone complet (collecte, validation, journalisation).
- Boîte à outils de décorateurs réutilisables.
- Checklist de revue pour code Python intermédiaire.

Glossaire technique approfondi

- **Encapsulation** : limitation de l'accès direct à l'état interne pour préserver les invariants métier.
- **Composition** : construction d'un comportement global en assemblant des objets spécialisés.
- **Héritage multiple** : combinaison de plusieurs classes parentes; son usage exige une lecture claire de la MRO.
- **MRO (Method Resolution Order)** : ordre exact de recherche d'une méthode quand plusieurs classes la définissent.
- **Descriptor** : objet implémentant `__get__` / `__set__` pour contrôler l'accès à un attribut.
- **Closure** : fonction qui capture des variables de son scope externe.
- **Idempotence** : propriété d'une opération pouvant être rejouée sans effet cumulé inattendu.
- **Backpressure** : mécanisme limitant le débit entrant pour éviter la saturation.
- **Cancellation** : arrêt contrôlé d'une tâche asynchrone pour préserver la cohérence applicative.
- **Structured concurrency** : modèle où les tâches vivent dans un périmètre explicite (ex: `TaskGroup`).

Détails techniques par bloc

POO avancée

- Utiliser `dataclass` pour les objets à forte composante données; préférer classes classiques quand la logique métier domine.
- Choisir `@property` pour exposer une API stable tout en gardant la validation interne.

- Implémenter `__repr__` pour le diagnostic, `__eq__` pour la comparaison métier, `__hash__` si usage en dictionnaire/set.
- Éviter les hiérarchies profondes: la composition réduit les régressions lors d'un refactor.
- Isoler les accès externes (DB/API) derrière des ports (`Protocol`) afin de tester le domaine sans I/O.

Décorateurs

- Conserver la signature de la fonction via `functools.wraps` pour la lisibilité et la compatibilité tooling.
- Utiliser des décorateurs paramétrés pour externaliser les politiques (timeout, retry, niveau de log).
- Distinguer clairement les responsabilités: un décorateur de validation ne doit pas gérer la persistance.
- Maîtriser l'ordre d'empilement des décorateurs: `@a @b f` équivaut à `a(b(f))`.
- Pour les coroutines, wrapper asynchrone obligatoire (`async def wrapper` + `await fn(...)`).

Async/await

- Utiliser `TaskGroup` pour borner le cycle de vie des tâches et simplifier la gestion des échecs.
- Définir un timeout explicite pour chaque dépendance réseau critique (`asyncio.wait_for`).
- Limiter la concurrence avec `Semaphore` pour éviter throttling et pics d'erreurs.
- Déléguer le code bloquant via `asyncio.to_thread` afin de ne pas bloquer l'event loop.
- Tracer latence, tentative, cause d'erreur pour rendre les incidents async investigables.

Pièges techniques fréquents et contremesures

- **Piège** : classes « dieu » cumulant logique, I/O et règles métier. **Contremesure** : séparer domaine, service et adaptateur.
- **Piège** : décorateurs opaques qui masquent l'erreur d'origine. **Contremesure** : remonter contexte + exception chainée.
- **Piège** : explosion de tâches async non bornées. **Contremesure** : stratégies de quota et batching.
- **Piège** : retry non idempotent créant des doublons métier. **Contremesure** : clé de déduplication + opérations idempotentes.
- **Piège** : dépendance implicite au global state. **Contremesure** : injection explicite de dépendances.